

Timetable Tactics: A Modular SMT Approach to the Exam Timetabling Problem

1 Introduction

Effective scheduling of exams is essential in academic institutions, requiring assignments of exams to specific rooms and times while meeting various constraints. This exam timetabling problem is NP-hard and involves satisfying multiple, often conflicting requirements (Müller, 2016). Addressing these challenges, this report presents a solution using Python and the Z3 SMT (Satisfiability Modulo Theories) solver (de Moura and Bjørner, 2008).

The solution leverages Z3's constraint-solving capabilities to handle modular constraints such as room capacities, non-overlapping schedules for students, and invigilator assignments. A graphical user interface (GUI) enables interactive test instance selection, with each instance executed in a subprocess to maintain GUI responsiveness, addressing Z3's limitations with multithreading.

The report further explores the problem formulation, alternative constraint methods, implementation details, and evaluation, alongside potential improvements and broader applications across various scheduling domains.

2 Problem Formulation

The exam timetabling problem requires assigning exams to specific rooms and time slots, ensuring that each assignment meets a set of predefined constraints. Let:

- $E = \{e_1, e_2, \dots, e_n\}$ be the set of exams,
- $R = \{r_1, r_2, \dots, r_m\}$ be the set of rooms,
- $T = \{t_1, t_2, \dots, t_k\}$ be the set of ordered time slots,
- $S = \{s_1, s_2, \dots, s_p\}$ be the set of students,
- $I = \{i_1, i_2, \dots, i_q\}$ be the set of invigilators,
- $c(r)$ be the capacity of room $r \in R$,
- $s(e) \subseteq S$ represent the set of students assigned to exam $e \in E$.

The following constraints are applied:

1. **Room and Slot Assignment:** Each exam must be assigned to exactly one room and exactly one time slot. Define $\text{Room}(e) \in R$ and $\text{Time}(e) \in T$ as the room and time slot assigned to exam e . Then, the assignment constraint can be expressed as:

$$\forall e \in E, \quad \exists r \in R, t \in T : \text{Room}(e) = r \text{ and } \text{Time}(e) = t.$$

2. **Non-Conflicting Room Assignments:** No two exams should be assigned to the same room in the same time slot. For any two exams $e, e' \in E$ (where $e \neq e'$), if $\text{Time}(e) = \text{Time}(e')$, then $\text{Room}(e) \neq \text{Room}(e')$. This constraint is formulated as:

$$\forall e, e' \in E, \quad (\text{Room}(e) = \text{Room}(e') \wedge \text{Time}(e) = \text{Time}(e')) \Rightarrow e = e'.$$

3. **Room Capacity Constraint:** The number of students assigned to an exam should not exceed the capacity of the room where it is scheduled. For each exam e , if $\text{Room}(e) = r$, then $|s(e)| \leq c(r)$. This constraint is expressed as:

$$\forall e \in E, \quad |s(e)| \leq c(\text{Room}(e)).$$

4. **No Consecutive Exams for Students:** Students should not have exams in consecutive time slots. For any two exams $e, e' \in E$ that share at least one student, if $\text{Time}(e) = t$, then $\text{Time}(e') \neq t \pm 1$. This constraint is represented as:

$$\forall e, e' \in E, \quad s(e) \cap s(e') \neq \emptyset \Rightarrow |\text{Time}(e) - \text{Time}(e')| > 1.$$

5. **Invigilator Assignment Constraint:** Each exam must have exactly one invigilator assigned. Define $\text{Invigilator}(e) \in I$ as the invigilator assigned to exam e . This requirement is expressed as:

$$\forall e \in E, \quad \exists i \in I : \text{Invigilator}(e) = i.$$

6. **No Consecutive Invigilator Assignments:** To prevent the same invigilator from being assigned to consecutive slots for different exams, for any two exams $e, e' \in E$, if $\text{Invigilator}(e) = \text{Invigilator}(e')$ and $e \neq e'$, then the time slots t and t' assigned to e and e' respectively should not be consecutive. This constraint can be expressed as:

$$\forall e, e' \in E, \quad \text{Invigilator}(e) = \text{Invigilator}(e') \Rightarrow |\text{Time}(e) - \text{Time}(e')| > 1.$$

These constraints are implemented using Z3's SMT functions (Nieuwenhuis et al., 2006), allowing for efficient verification of satisfiability. The modular approach enables constraints to be dynamically applied based on instance attributes.

3 Alternative Formulations

While the initial formulations implemented for each constraint ensure the satisfiability of the exam timetabling problem, alternative approaches may simplify or optimise the constraint-solving process. The following alternative formulations address the main constraints while aiming to reduce complexity or enhance readability.

3.1 Alternative Formulation for Room and Slot Assignment (Constraint 1 & 2)

The primary formulation uses **ForAll** and **Exists** quantifiers to ensure that each exam is assigned to exactly one room and one slot. However, an alternative approach could simplify this by introducing a composite variable for each exam representing both room and slot. Let $\text{Assignment}(e)$ be a composite pair (r, t) where $r \in R$ and $t \in T$. We then ensure that:

$$\forall e \in E, \quad \exists \text{Assignment}(e) : \text{Assignment}(e) = (r, t).$$

This formulation consolidates room and slot assignments, reducing the number of separate declarations for rooms and slots. Although it may be less flexible in capturing more granular conditions on room or slot constraints, this simplification may improve performance for larger problem instances by reducing the number of individual constraints Z3 must evaluate. However, using a composite slot-room pair could increase the complexity in managing room capacities and may reduce overall solution clarity (Jovanović and de Moura, 2011).

3.2 Alternative Formulation for Room Capacity (Constraint 3)

In the original approach, the room capacity constraint uses implications to check that each exam's student count does not exceed room capacity. An alternative approach could precompute feasible room options based on the student count for each exam. Let:

$$\text{FeasibleRooms}(e) = \{r \in R : |s(e)| \leq c(r)\}.$$

Then, for each exam, the constraint would only assign a room from this precomputed subset:

$$\forall e \in E, \quad \text{Room}(e) \in \text{FeasibleRooms}(e).$$

This reduces the need for runtime checks within the solver, as Z3 only considers feasible room options during its evaluation.

3.3 Alternative Formulation for No Consecutive Exams (Constraint 4)

The initial approach prohibits consecutive exams for students by imposing constraints on all exams for each student. As an alternative, we could precompute consecutive exam pairs for each student and apply constraints only to these specific pairs. Let $\text{ConsecutivePairs}(s)$ represent all pairs of exams (e, e') for student s that occur in consecutive time slots. The constraint could then be reformulated as:

$$\forall s \in S, \forall (e, e') \in \text{ConsecutivePairs}(s), \quad |\text{Time}(e) - \text{Time}(e')| > 1.$$

This reduces the number of constraints, focusing only on consecutive slot pairs rather than enforcing this constraint across all possible exams for each student.

3.4 Alternative Formulation for Invigilator Assignment (Constraint 5)

The invigilator assignment constraint initially requires each exam to have exactly one invigilator. An alternative approach could involve grouping exams by time slot and assigning invigilators to these groups. Let:

$$\text{AssignedInvigilator}(t) = \{i \in I : i \text{ is assigned to a time slot } t\}.$$

Then for each exam e occurring in time slot t , we enforce that:

$$\text{Invigilator}(e) \in \text{AssignedInvigilator}(t).$$

This allows invigilators to be assigned based on time slots rather than on a per-exam basis, reducing the number of constraints and potentially improving performance.

3.5 Alternative Formulation for No Consecutive Invigilator Assignments (Constraint 6)

The original constraint prevents invigilators from being assigned to consecutive exams by checking time slot adjacency. An alternative approach could use a sequence-based constraint to track invigilator assignments over consecutive slots directly. Define a variable $\text{NextSlot}(i, t)$ that maps an invigilator i to their next assigned slot following t . Then, for each invigilator i assigned to exams e and e' in consecutive time slots t and $t + 1$, we enforce:

$$\text{NextSlot}(i, t) \neq t + 1.$$

This formulation tracks invigilator assignments across time slots in sequence, potentially simplifying adjacency constraints by ensuring that consecutive time slots are never assigned to the same invigilator without needing to check all pairs.

These alternative formulations aim to provide different perspectives on implementing each constraint, potentially improving efficiency or simplifying the solution by reducing the computational load required by Z3.

4 Implementation

The implementation of the exam timetabling solution uses Python and the Z3 solver for constraint satisfaction, with a graphical user interface (GUI) enabling users to select problem instances and view the results. A subprocess approach is employed to avoid threading issues with Z3 in multi-threaded GUI environments, allowing multiple instances to be solved concurrently without impacting the GUI's responsiveness. Additionally, the solver finds multiple solutions by iteratively excluding each solution from future checks, allowing for alternative results where possible.

4.1 Setup and Execution

The steps to set up and run the solver are as follows:

1. **Create a Virtual Environment:** First, create a virtual environment in Python to ensure dependencies are isolated and compatible. This can be done with:

```
python -m venv env
```

2. **Install Required Dependencies:** Install all required packages using the provided `requirements.txt` file:

```
pip install -r requirements.txt
```

3. **Run the Solver:** To execute the solver, either of the following main files can be run:

- `main.py`: Runs the solver without a GUI, processing all instances in the specified folder.
- `gui.py`: Launches a GUI interface where users can select specific test instances, initiate the solver, and view results in a user-friendly format.

4.2 Logic of the Solver

The solver's logic is designed to apply constraints to an instance, find solutions, and output the results. Key steps in this process include instance preparation, constraint application, solution generation, and result formatting.

- **Instance Setup and Declarations:** The solver begins by creating an instance of the `Solver` class in Z3, which will manage all constraints and solution checks. Using the attributes of the instance, the `Declarations` class generates the necessary variables (e.g., exams, rooms, time slots) and functions for constraints. This structure enables constraints to be dynamically applied based on instance attributes.
- **Applying Constraints:** The `ConstraintManager` class manages and applies constraints to the solver. Each constraint is defined in a separate class (e.g., `ExamAssignmentConstraint` or `RoomCapacityConstraint`), with an `add_to_solver` method that adds its specific logic to the solver. This modular design facilitates easy modification and extension of constraints as needed.
- **Constraint Setup and Application:** An array of constraints, such as `RangeConstraint`, `StudentAssignmentsConstraint`, and `ExamAssignmentConstraint`, is set up and managed by `ConstraintManager`, which applies each constraint to the solver in turn. This process ensures that all required constraints are enforced before solution generation.

4.3 Detailed Solver Logic

The main logic of the solver is encapsulated in the `solve` function, which operates as follows:

1. **Initialise Solver and Declarations:** A new Z3 solver instance is created, and relevant declarations are generated using the `Declarations` class based on the attributes of the provided instance.
2. **Constraint Setup and Application:** An array of constraints, such as `RangeConstraint`, `StudentAssignmentsConstraint`, and `ExamAssignmentConstraint`, is defined and managed by `ConstraintManager`. Each constraint is applied to the solver in turn.
3. **Solution Loop:** A loop is initiated to check for satisfiability:
 - **Model Extraction:** If the instance is satisfiable, the current model is retrieved and evaluated for the first solution. The assignments of exams to rooms and slots are recorded in `result_text` to display which exam is assigned to each room and slot.

- **Exclusion of Current Model:** The current model is then excluded by adding a negation of the current assignments to the solver. This process enables finding additional solutions by ensuring the solver does not return the same model in subsequent checks.
 - **Solution Count and Timing:** After each solution, `model_count` is incremented, and the time for finding the first solution is recorded. If the solution count reaches 1,000, the loop terminates.
4. **Result Formatting:** If no solutions are found, `unsat` is recorded in `result_text`. The function then appends the total number of solutions found (or `model_count - 1` for alternatives) and the elapsed times for the first and total solutions. The formatted results are returned to the main execution context or displayed in the GUI.

4.4 Concurrency and GUI Integration

Due to Z3's limitations with threading in GUI applications, the solver uses a subprocess for each instance. This setup prevents blocking the GUI, allowing users to continue interacting with the interface while solutions are computed. The `gui.py` script provides an accessible interface, enabling users to:

- Select one or multiple instances for evaluation.
- Start the solver process and view live results in the GUI's output area.
- View results for each solution, including time elapsed for the first solution and total elapsed time.

This architecture provides flexibility and responsiveness in the user interface, making it easier to work with multiple instances simultaneously without GUI lag.

5 Evaluation

The solver was evaluated using a set of 20 test instances, each containing distinct configurations to assess the solver's performance and accuracy across varied scenarios. The results confirmed that the solver could handle all test instances efficiently, generating correct solutions within acceptable time limits.

5.1 Performance and Solution Time

For each test instance, the solution time was tracked and displayed in milliseconds, providing insights into the solver's efficiency. The solver reports the following metrics:

- **First Solution Time:** The time taken to find the first solution for each instance is measured and displayed in milliseconds, allowing a direct view of the solver's initial response time.
- **Total Solution Time:** This metric captures the total time taken to explore all solutions for a given instance. It is especially useful for instances with multiple possible solutions, as it indicates the cumulative time required to identify every possible outcome that satisfies the constraints.
- **Alternative Solutions:** The solver's design allows it to identify alternative solutions by iteratively excluding each current model from further checks. For each instance, the number of alternative solutions is displayed, with an additional indicator if the number of alternatives exceeds a threshold (such as 1,000 solutions).

5.2 Correctness and Completeness

The solver was tested on the 20 instances with the four mandatory constraints: Room and Slot Assignment, Non-Conflicting Room Assignments, Room Capacity, and No Consecutive Exams for Students. Across all instances, the solver produced correct results that adhered to each constraint, validating the core functionality of the solution. Additional testing included the optional invigilator assignment constraints, which the solver successfully enforced on the same instances. This demonstrated the solver's adaptability in handling both mandatory and optional constraints.

5.3 GUI Execution and Subprocess Overheads

The solver can also be run through a graphical user interface (GUI), providing an accessible way for users to select instances, initiate the solver, and view results in real time. However, testing showed a slight increase in solution time when the solver is run within the GUI. This increase is due to the use of subprocesses to manage each instance independently, ensuring the GUI remains responsive while solving. Each instance runs as a separate subprocess to prevent threading issues with Z3, which can arise in GUI environments. This subprocess approach, while effective in maintaining the GUI's responsiveness, introduces minor additional overhead due to the need to initialise each subprocess and manage inter-process communication.

5.4 Summary of Results

Overall, the solver demonstrated strong performance across all test cases, successfully handling both mandatory and optional constraints and offering multiple solutions where available. The time-tracking features, alongside the reporting of alternative solutions, provide detailed insights into the solver's efficiency and robustness. Although the GUI introduces a small overhead due to subprocess execution, it offers a convenient and user-friendly interface for managing and solving multiple instances seamlessly.

6 Discussion

This implementation successfully addresses the complex requirements of the exam timetabling problem, ensuring that all constraints are met while providing a versatile solution adaptable to various scenarios. The solver's modular code structure and GUI interface enhance user interaction, making it easy to select and solve multiple problem instances while viewing results in real time. The use of subprocesses for each instance within the GUI environment effectively circumvents multithreading limitations in Z3, although this requires careful handling of inter-process communication to maintain performance. Z3's efficient constraint satisfaction checking allows for exploration of multiple solutions, showcasing the robustness and flexibility of the solver.

6.1 Application Overview

The exam timetabling solution is designed to meet the needs of academic institutions and examination boards that must assign exams to specific rooms and time slots while satisfying various constraints, such as room capacities and non-overlapping schedules for students and invigilators. The solver is structured to handle different types of constraints dynamically, allowing for both mandatory and optional conditions based on instance attributes. This adaptability makes it suitable for a wide range of applications, including scenarios where additional requirements (such as invigilator scheduling or custom room allocations) are necessary.

The GUI component is especially valuable for users who may not be familiar with coding, as it provides an intuitive interface for selecting instances, initiating the solver, and viewing results. By offering real-time feedback on solution times and alternate models, the GUI contributes significantly to the overall usability of the application. Additionally, the solver's modularity means that new constraints or adjustments to existing ones can be easily implemented, making the solution extensible and future-proof. This capability is critical for institutions that may need to incorporate changing requirements over time, such as adjusting room capacities or introducing new constraints related to student welfare.

6.2 Extraordinary Features

The solver incorporates several extraordinary features that enhance its robustness, flexibility, and user accessibility. First, the code has been fully refactored and modularised, with each constraint encapsulated in a dedicated class. Utility functions handle additional tasks, such as time tracking and input processing, ensuring that the code remains organised, readable, and maintainable. This modularisation allows for straightforward modifications, enabling constraints to be added or adjusted without affecting the overall structure.

A clear separation of concerns has been implemented throughout the code. For instance, the initialisation of instances is solely managed by the `Instance` class, while attribute reading is delegated to a separate utility function. This separation results in a clean, focused design where each component has a specific role, reducing potential dependencies and making it easier to troubleshoot or expand upon.

Another significant feature is the dynamic initialisation of declarations and constraints. The solver checks for the presence of required attributes in each problem instance, and if specific attributes are absent, related constraints are omitted. This design ensures that the solver only processes relevant constraints, enhancing performance and adaptability to different instance types.

The GUI implementation adds further value by allowing users to solve single or multiple instances with ease. This graphical interface provides a user-friendly experience, making the solver accessible to individuals who may not have programming experience. Users can select instances, initiate the solver, and view the results directly within the GUI, all while preserving the responsiveness of the interface.

To address multithreading limitations with Z3 in GUI environments, the solver employs a subprocess approach for each instance. By running each instance in its own subprocess, the solver ensures that the GUI remains responsive even when multiple instances are being processed concurrently. Although this adds minor overhead, it significantly enhances the usability of the application.

Additionally, utility functions have been developed to process input files and track time accurately. These functions allow time differences to be measured in various units, such as milliseconds and seconds, enabling detailed performance evaluation across instances. The solver's capacity to find multiple solutions is also noteworthy; by iteratively excluding each solution from further checks, the solver can display all valid alternatives for each instance, providing insight into the flexibility of possible exam schedules.

6.3 Future Enhancements and Potential Use Cases

While the solver performs effectively in its current state, several future enhancements could further extend its functionality. One promising avenue is the integration of hyper-heuristics, which could enable the solver to adaptively choose from a set of heuristic approaches depending on the characteristics of the instance being solved. By adjusting the heuristic selection based on real-time performance feedback, the solver could potentially achieve faster and more optimised solutions for complex, large-scale instances. Hyper-heuristics are particularly beneficial for exam timetabling, as they allow the system to dynamically adjust to variations in problem complexity without requiring manual tuning.

Additionally, integrating machine learning algorithms for constraint relaxation or prioritisation could facilitate automated decision-making in scenarios where certain constraints cannot be fully satisfied. This approach would allow the solver to assess which constraints might be relaxed to provide feasible solutions when the problem becomes over-constrained, ultimately enhancing the solver's flexibility.

Incorporating advanced optimisation techniques could also help minimise solution time for large instances, as well as reduce the number of room changes required by students. Such optimisation could target specific metrics, such as minimising scheduling conflicts or maximising room usage efficiency, to produce higher-quality solutions that better meet institutional needs.

The solver's modular and adaptable design makes it suitable for broader applications beyond exam timetabling. For instance, it could be adapted for conference scheduling, where multiple sessions need to be assigned to rooms and time slots while respecting constraints related to speaker availability and room capacities. Other potential applications include scheduling shift work in organisations, managing resource allocation in manufacturing, or any scenario requiring complex scheduling with strict constraints. These features, combined with its extensibility, make the solver a valuable tool for both academic and industrial use cases.

Overall, this implementation represents a highly adaptable and robust approach to the exam timetabling problem. The combination of modularity, a user-friendly GUI, and dynamic constraint management ensures that it meets current needs while remaining flexible enough for future adaptation and improvement.

7 Conclusion

7.1 Summary of Contributions

The developed exam timetabling solver effectively addresses the complexities of scheduling under various constraints by leveraging Python and the Z3 SMT solver. With modular constraint handling, the solver is capable of assigning exams to specific rooms and times while meeting essential requirements, such as room capacity limits, non-overlapping schedules for students, and invigilator assignments. This modular design also enables the solver to dynamically adjust constraints based on instance attributes, enhancing adaptability for diverse scheduling needs.

7.2 Limitations

While the solver demonstrates strong performance, certain limitations of Z3 impact its expressiveness and functionality:

- **Lack of “There Exists Exactly One” ($\exists!$) Operator:** Z3 does not have a native representation for this operator, requiring workaround formulations that can increase the complexity of constraint modeling.
- **Absence of Universal Set Representation:** Z3 lacks a direct way to define universal sets for variables. For instance, rather than expressing $e \in E$ directly, a workaround is needed, such as representing it with `Exam.Range(e)` to define the bounds of variable e .
- **No Default Mechanism for Multiple Solutions:** Z3 does not inherently support the retrieval of all possible solutions. To address this, an exclusion mechanism is used to iteratively find unique solutions, although this approach may affect performance in instances that require extensive solution exploration.

7.3 Future Work

Potential improvements could focus on integrating heuristic or optimization techniques to increase scalability and efficiency, particularly in large instances. Furthermore, exploring alternative solvers or solver extensions that better support these limitations may enhance the solver’s applicability and performance in more complex scheduling scenarios.

7.4 Conclusion

Overall, this exam timetabling solver demonstrates the potential of SMT solvers in managing complex scheduling tasks. Through modular design and flexible constraint application, the solution provides a robust and adaptable approach to timetabling and opens pathways for application in other scheduling domains.

References

- de Moura, L. and Bjørner, N. (2008). Z3: An efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer Berlin Heidelberg.
- Jovanović, D. and de Moura, L. (2011). Solving non-linear arithmetic. In *Proceedings of the 6th International Joint Conference on Automated Reasoning*, pages 339–354.
- Müller, T. (2016). Real-life examination timetabling. *Journal of Scheduling*, 19:257–270.
- Nieuwenhuis, R., Oliveras, A., and Tinelli, C. (2006). Solving sat and sat modulo theories: From an abstract davis–putnam–logemann–loveland procedure to dpll(t). *Journal of the ACM (JACM)*, 53(6):937–977.